AWS Lab: Application Load Balancer

Date: 2nd November 2026

1. Objective

To configure and demonstrate application-level load balancing on Amazon Web Services (AWS) by deploying two EC2 web server instances in different Availability Zones, registering them with a target group, and routing incoming HTTP requests through an internet-facing Application Load Balancer (ALB). The practical verifies that requests sent to the ALB can be served by either backend web server.

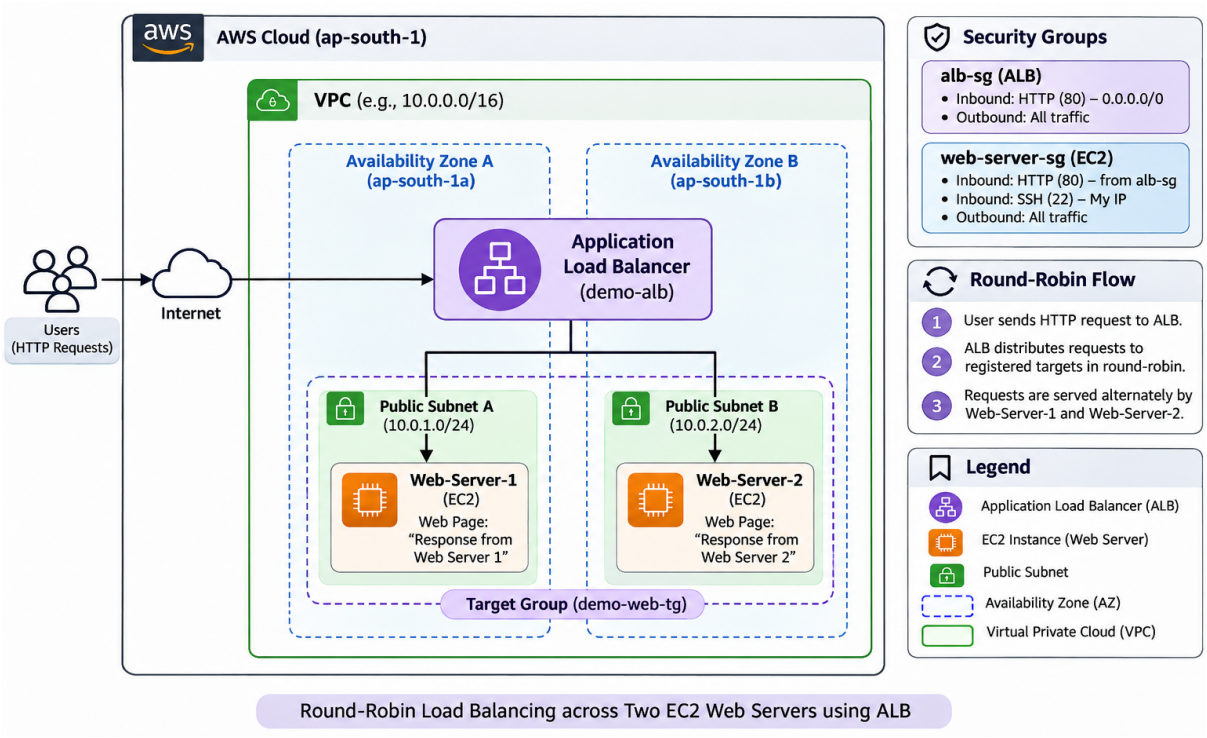


Figure 1: Topology of the Application Load Balancer architecture.

2. Theory

2.1 Load Balancing

Load balancing is the process of distributing incoming network or application traffic across multiple backend servers. Instead of sending every request to a single server, a

load balancer selects an available target. This helps improve application availability, scalability, and fault tolerance.

2.2 Application Load Balancer

An Application Load Balancer is an AWS Elastic Load Balancing service that operates at the application layer (Layer 7) of the OSI model. It supports HTTP and HTTPS traffic and can route requests to targets such as EC2 instances, containers, and IP addresses. A listener accepts client connections on a configured protocol and port, then applies listener rules to forward requests to a target group.

2.3 Target Group and Health Checks

A target group is a logical collection of backend targets. In this experiment, the target type is *Instances*, and both EC2 web servers are registered on HTTP port 80. The ALB periodically performs health checks using the configured path (`/`). Only targets that pass health checks are considered healthy and eligible to receive normal traffic.

2.4 Round-Robin Algorithm

The default routing algorithm for an Application Load Balancer target group is round robin. It distributes requests among healthy registered targets in turn. With two healthy targets, successive independent requests will generally be spread across both servers. However, the observed order is not a guarantee of strict alternation in every test: connection reuse, request timing, target health, and changes in target-group attributes can affect the visible sequence.

2.5 Security Groups

Security groups act as virtual firewalls for AWS resources. The ALB security group should permit inbound HTTP traffic from clients. The EC2 security group should permit HTTP traffic from the ALB security group, rather than from the entire internet. SSH access should be restricted to the administrator's public IP address.

3. Requirements

- An active AWS account with permission to create VPC security groups, EC2 instances, target groups, and load balancers.
- One VPC with at least two public subnets in different Availability Zones.
- Internet Gateway attached to the VPC and suitable subnet route tables (for an internet-facing ALB).
- Two Amazon Linux 2023 EC2 instances.
- A web server installed on each EC2 instance.
- A web browser or terminal with `curl` for testing.

4. Procedure

4.1 Step 1: Create the ALB Security Group

1. Open the AWS VPC console and navigate to **Security Groups**.
2. Choose **Create security group**.
3. Enter the security group name as `alb-sg`.
4. Select the VPC in which the load balancer and EC2 instances will be deployed.
5. Add an inbound rule: Type **HTTP**, port **80**, source `0.0.0.0/0`.
6. Keep the default outbound rule allowing all traffic, then create the security group.

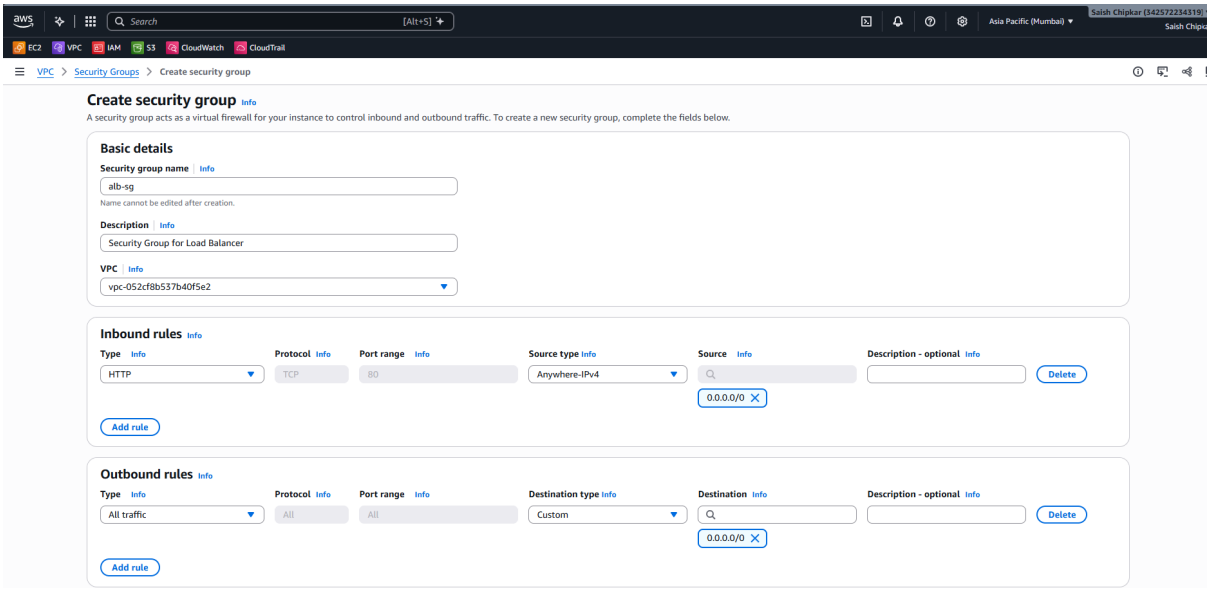


Figure 2: Creation of the ALB security group `alb-sg`, allowing inbound HTTP traffic.

4.2 Step 2: Create the EC2 Web Server Security Group

1. Create another security group named `web-server-sg` in the same VPC.
2. Add an HTTP inbound rule on port 80 with the source set to the ALB security group (`alb-sg`).
3. Add an SSH inbound rule on port 22 with the source set to **My IP**.
4. Keep the default outbound rule and create the security group.

Important security note: The backend HTTP rule must reference the ALB security group. The ALB itself must use `alb-sg`; do not expose the EC2 instances directly to `0.0.0.0/0` for HTTP.

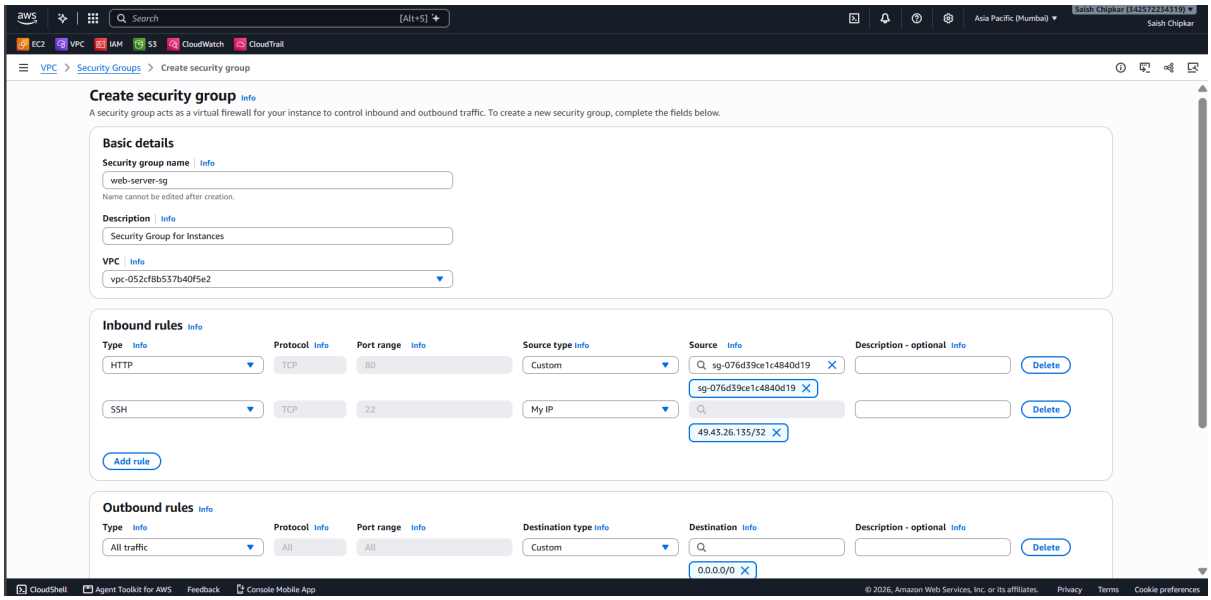


Figure 3: Creation of the backend EC2 security group with HTTP and SSH inbound rules.

4.3 Step 3: Launch the First EC2 Web Server

1. Navigate to **EC2** → **Instances** → **Launch instances**.
2. Set the instance name to `web-server-1`.
3. Select an Amazon Linux 2023 AMI and a suitable instance type.
4. Select the required VPC and Subnet A in the first Availability Zone.
5. Select the existing security group `web-server-sg`.
6. In **Advanced details**, enter the following User Data script. This script installs Nginx and creates a page that identifies the first server.

Listing 1: User Data script for Web Server 1

```
#!/bin/bash
dnf update -y
dnf install -y nginx
systemctl enable --now nginx
echo "<h1>Server 1 at AZ 1</h1>" > /usr/share/nginx/html/index.html
```

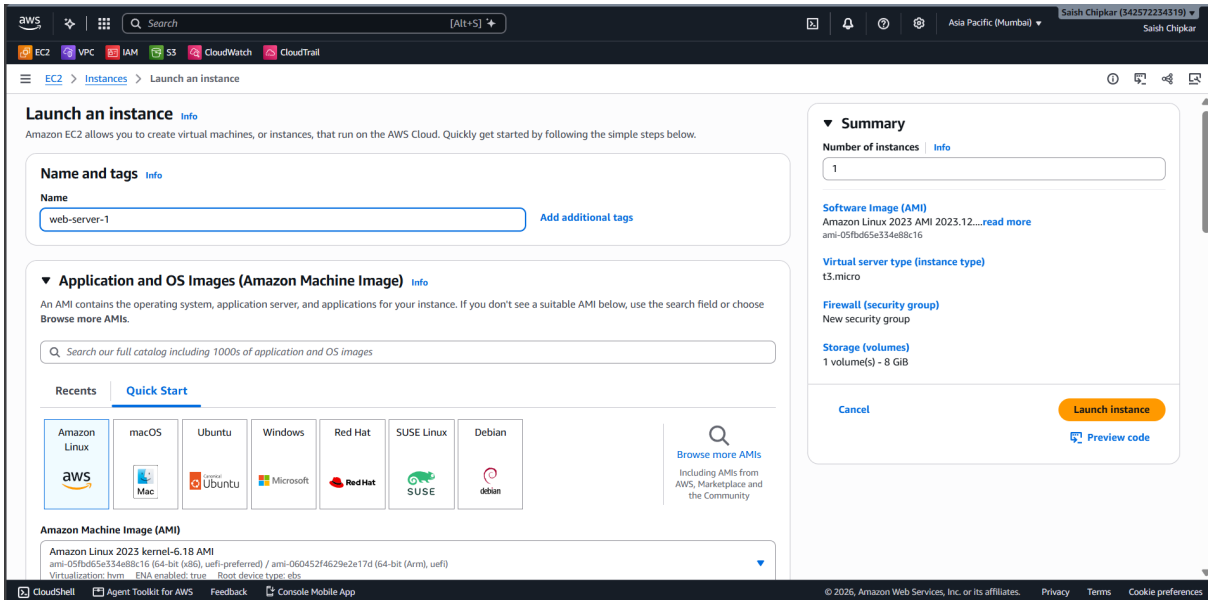


Figure 4: Launching the first EC2 instance using the Amazon Linux 2023 AMI.

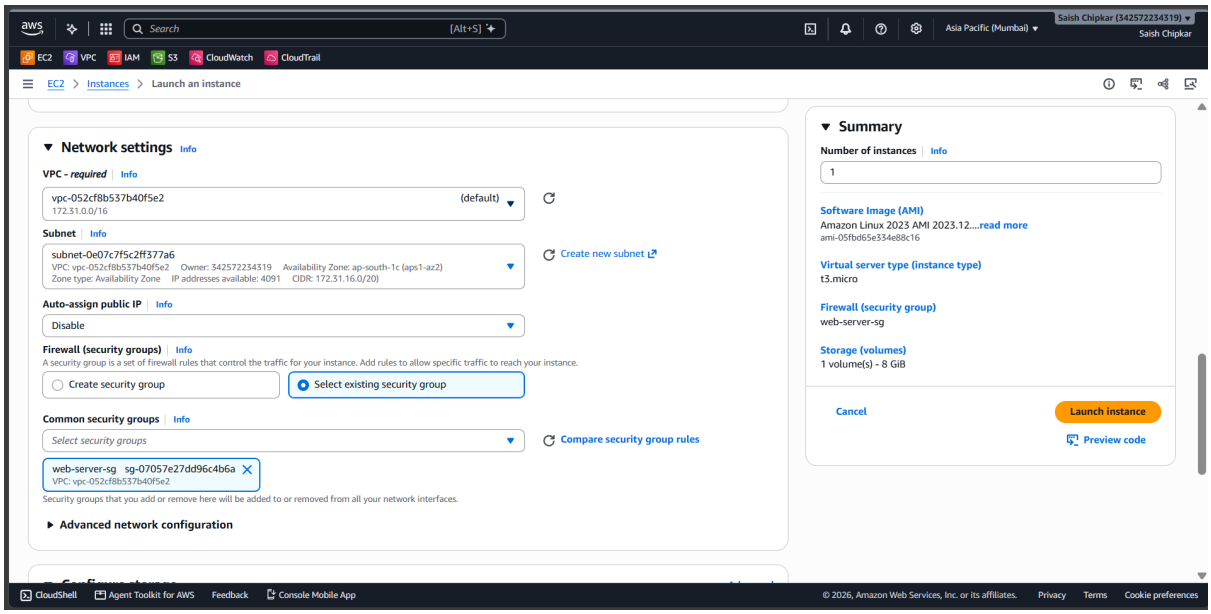


Figure 5: Selecting the first subnet and attaching `web-server-sg`.

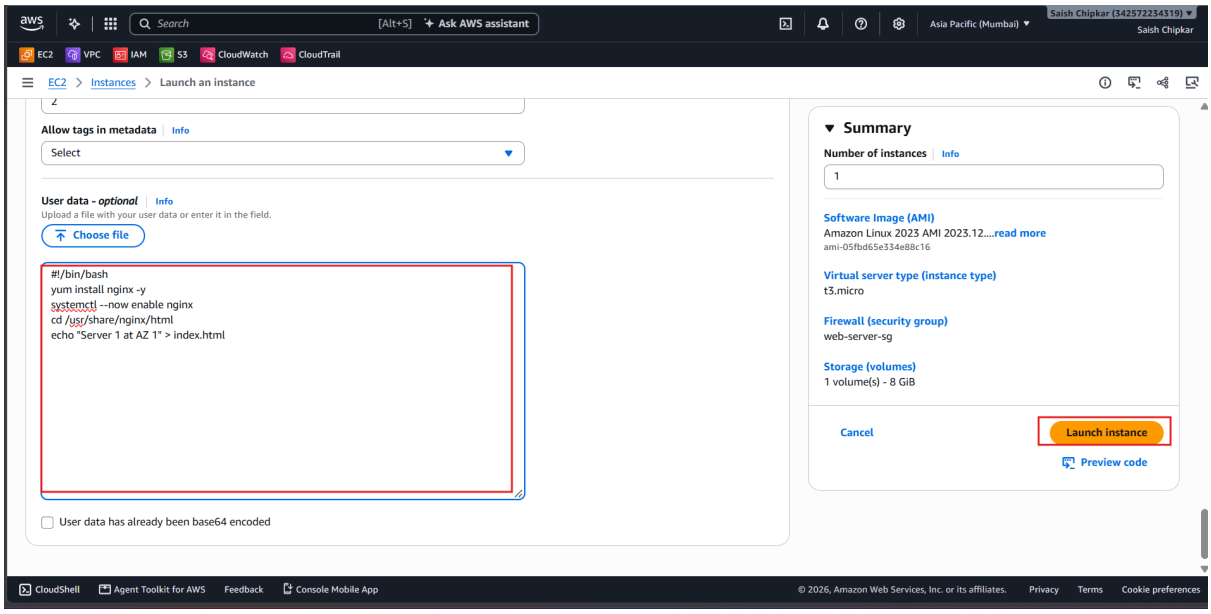


Figure 6: Entering the User Data script for the first Nginx web server.

4.4 Step 4: Launch the Second EC2 Web Server

1. Launch a second EC2 instance and set its name to `web-server-2`.
2. Select the same Amazon Linux 2023 AMI and a suitable instance type.
3. Select Subnet B in a different Availability Zone from the first instance.
4. Attach the `web-server-sg` security group.
5. Enter the following User Data script to create a distinct web page.

Listing 2: User Data script for Web Server 2

```
#!/bin/bash
dnf update -y
dnf install -y nginx
systemctl enable --now nginx
echo "<h1>Server 2 at AZ 2</h1>" > /usr/share/nginx/html/index.html
```

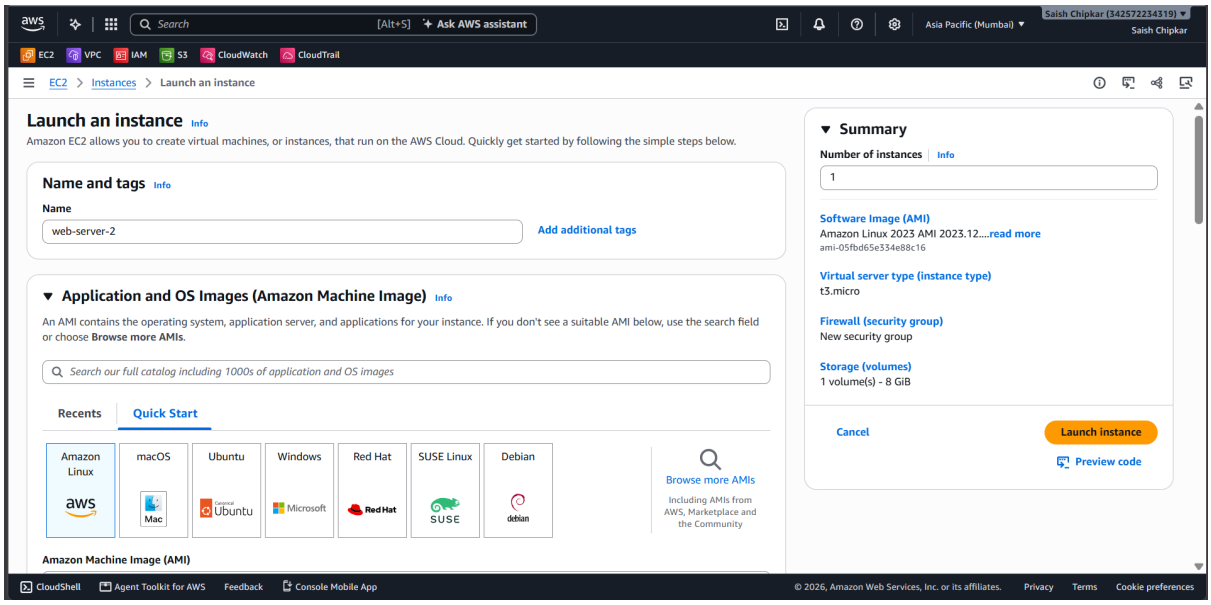


Figure 7: Launching the second EC2 instance.

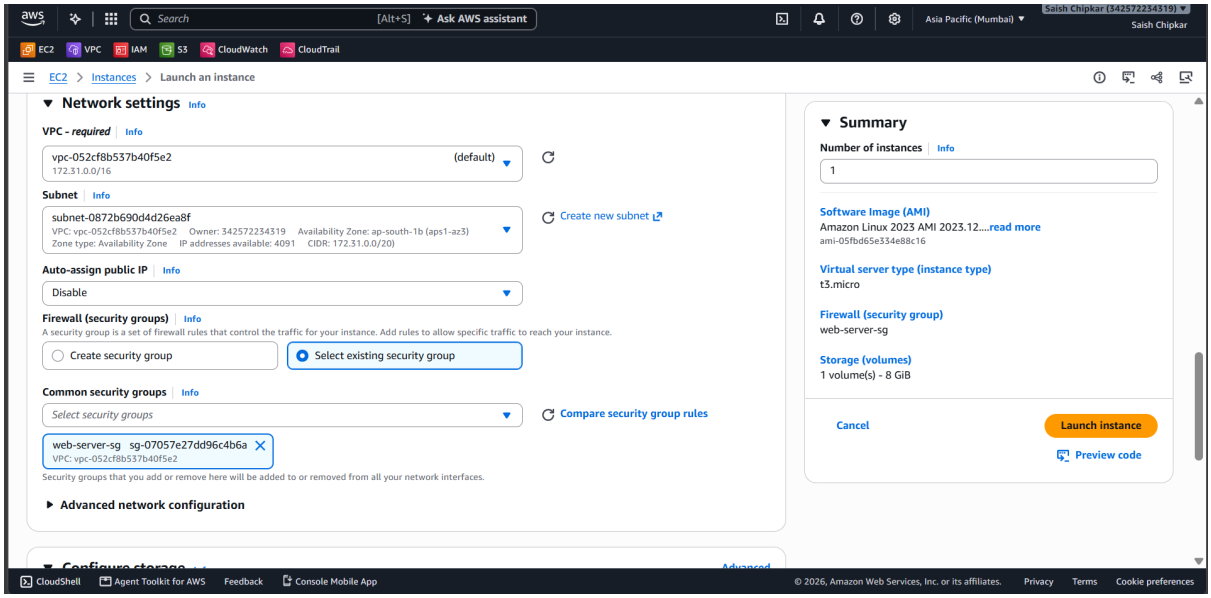


Figure 8: Selecting the second subnet and attaching the backend security group.

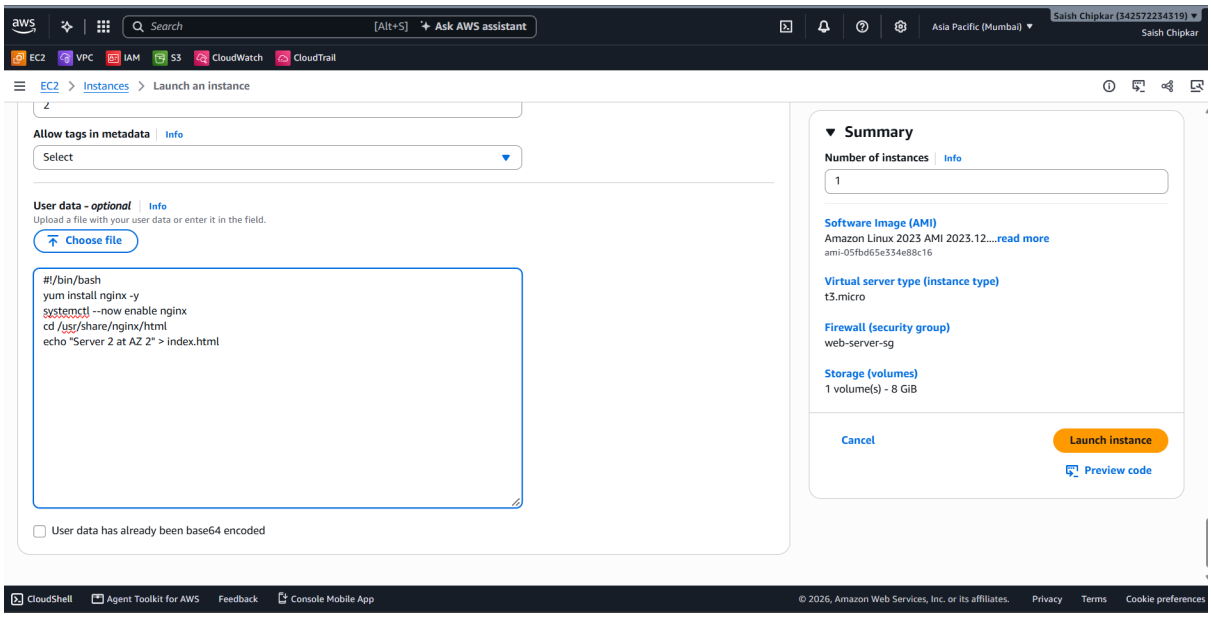


Figure 9: Entering the User Data script for the second Nginx web server.

Note: The supplied screenshots show Nginx being installed with `dnf`. The scripts above match that configuration. If Apache HTTP Server is preferred instead, use `httpd` and its corresponding document root, but do not mix the two configurations.

4.5 Step 5: Create the Target Group

1. Open **EC2** → **Target Groups**.
2. Choose **Create target group**.
3. Select **Instances** as the target type.
4. Enter `demo-web-tg` as the target group name.
5. Set the protocol to **HTTP**, port to **80**, and select the correct VPC.
6. Keep the health check protocol as HTTP and set the health check path to `/`.
7. Continue to the target registration page.

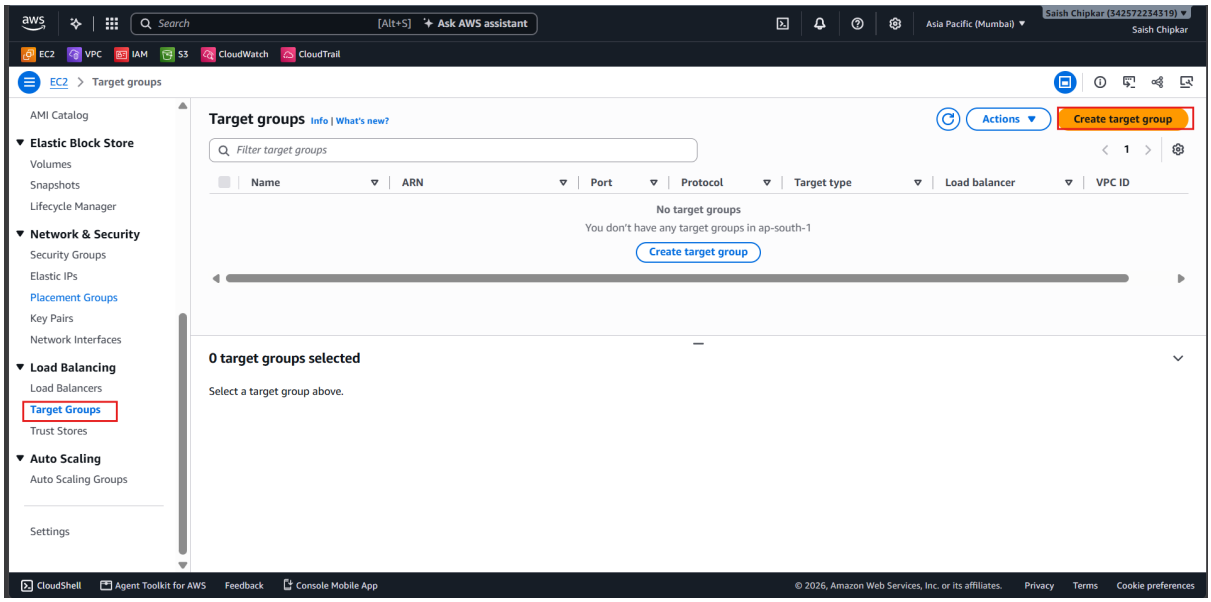


Figure 10: Opening the Target Groups section in the EC2 console.

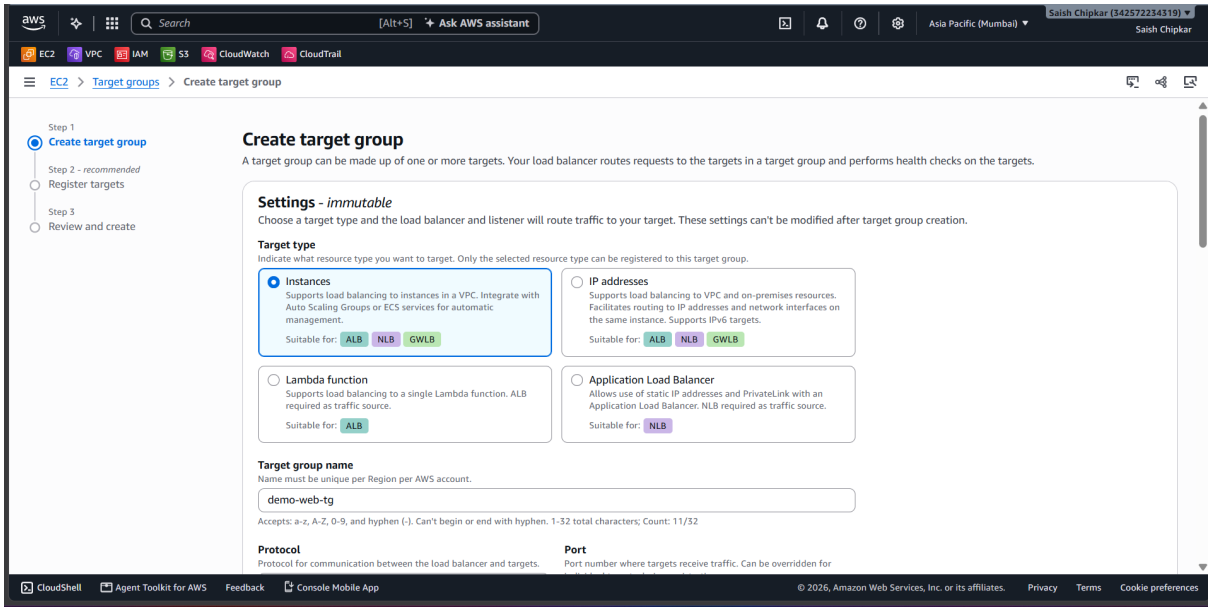


Figure 11: Selecting the instance target type and entering the target group name.

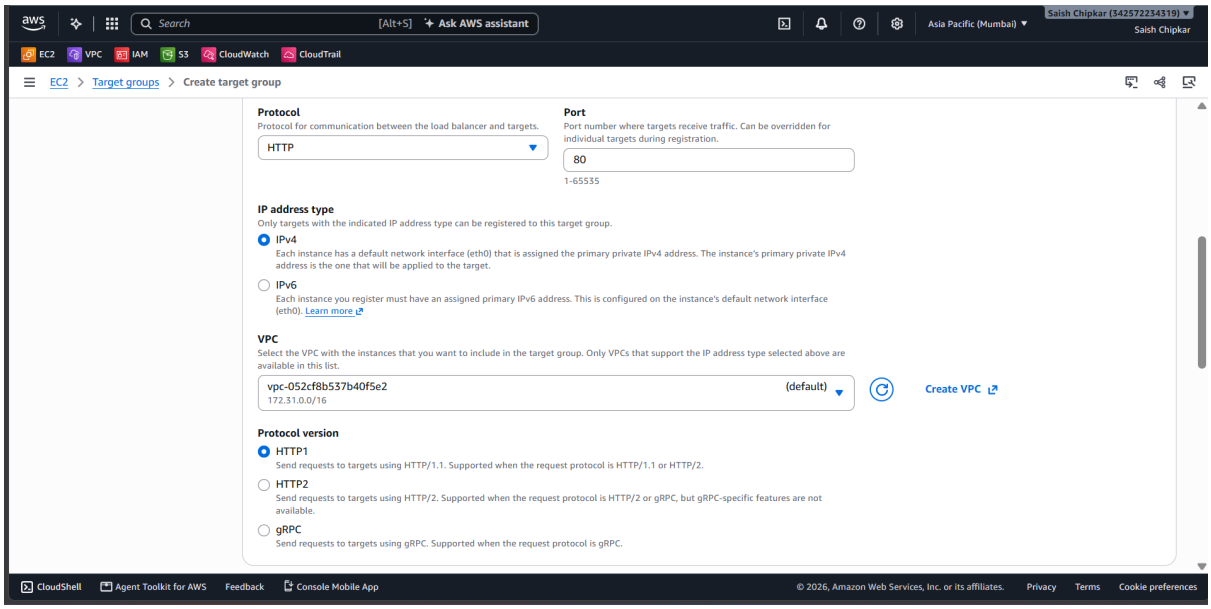


Figure 12: Configuring HTTP on port 80 and selecting the VPC.

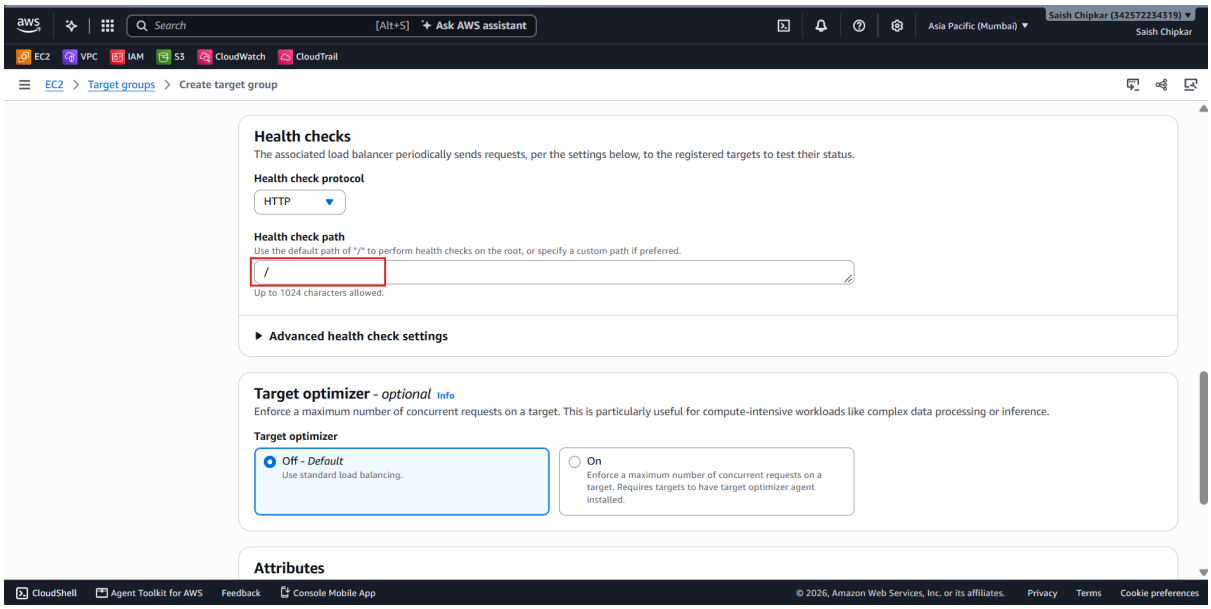


Figure 13: Configuring the HTTP health check path as `/`.

8. Select both `web-server-1` and `web-server-2`.

9. Confirm that the registered target port is 80 and choose **Include as pending below**.

10. Review the target group configuration and create the target group.

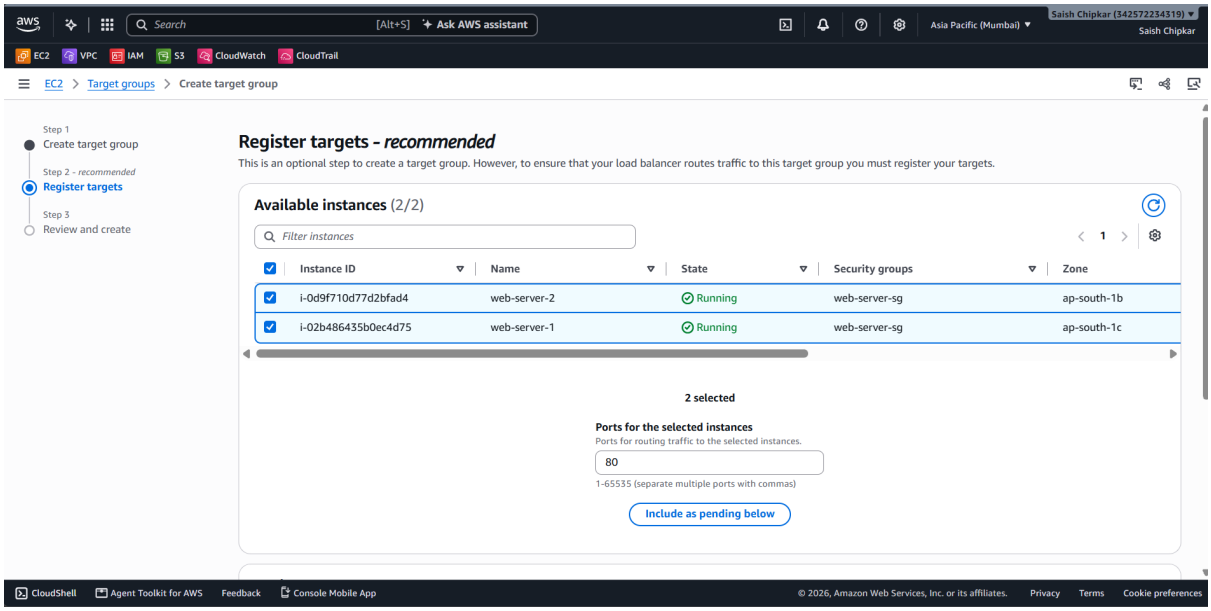


Figure 14: Selecting both EC2 instances for registration in the target group.

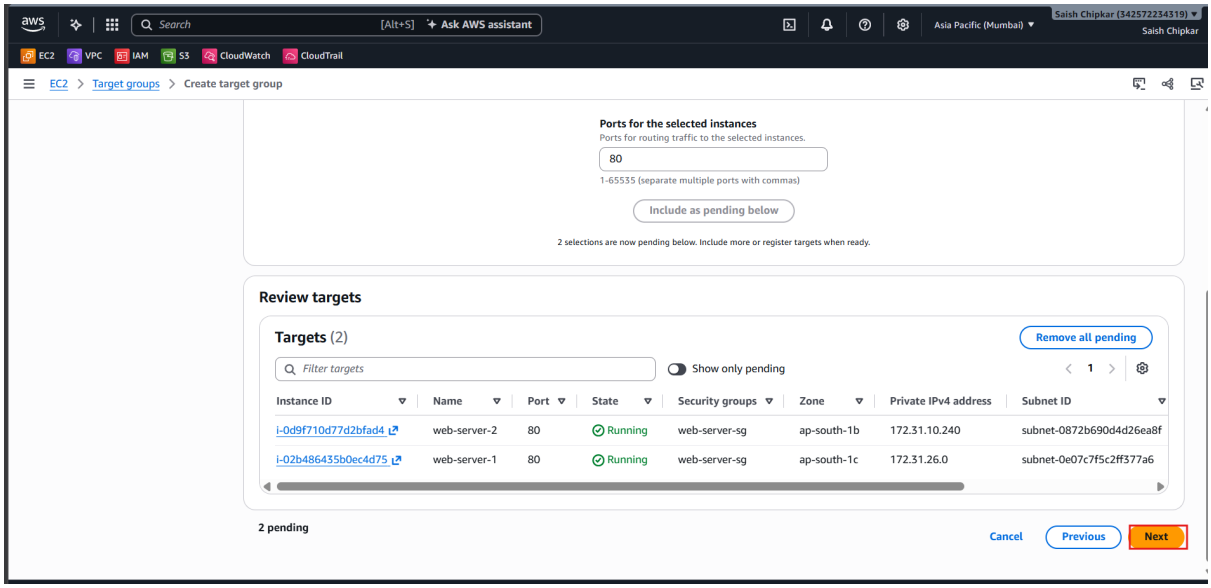


Figure 15: Reviewing the target group settings and registered targets before creation.

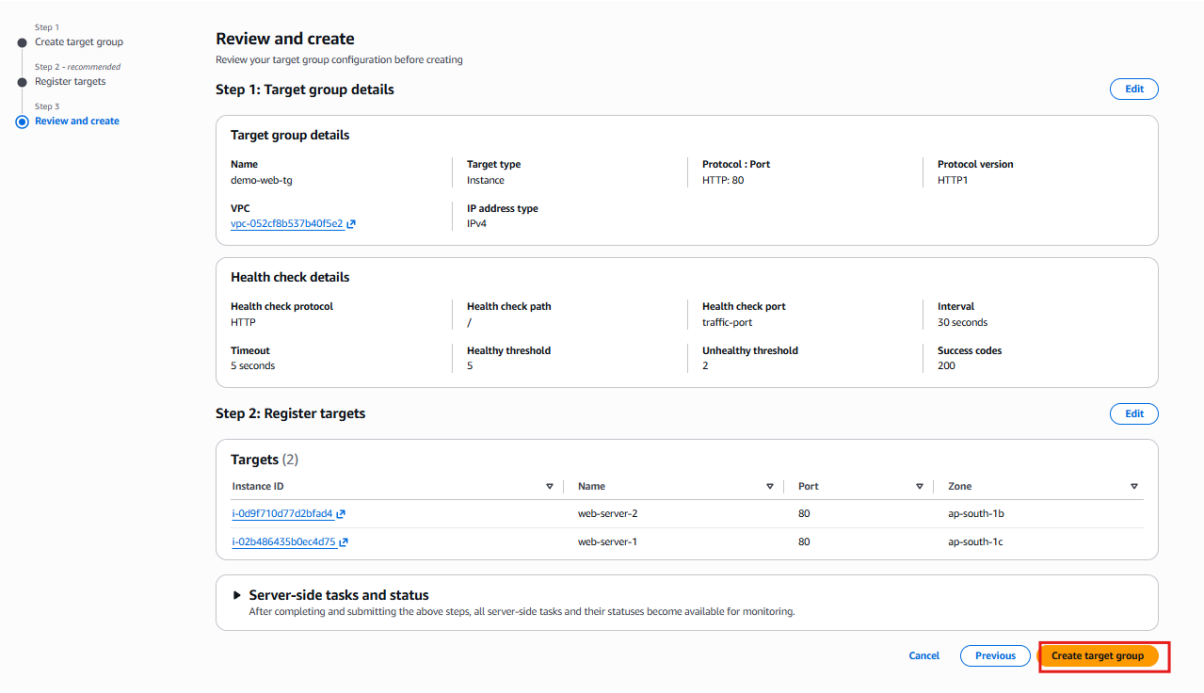


Figure 16: Target group details after creation. The targets may initially appear as unused or pending until the load balancer is associated and health checks complete.

4.6 Step 6: Create the Application Load Balancer

1. Navigate to **EC2** → **Load Balancers** and choose **Create load balancer**.
2. Select **Application Load Balancer**.
3. Enter `demo-alb` as the load balancer name.
4. Select **Internet-facing** as the scheme and **IPv4** as the IP address type.
5. Select the same VPC and choose public subnets in at least two Availability Zones. The ALB needs suitable routes to an Internet Gateway for public internet access.
6. Under Security groups, attach `alb-sg`. Ensure the ALB security group permits inbound HTTP on port 80 from client networks.
7. Configure a listener using protocol **HTTP** and port **80**.
8. Set the default listener action to **Forward to target groups** and select `demo-web-tg`.
9. Review the configuration and create the load balancer.

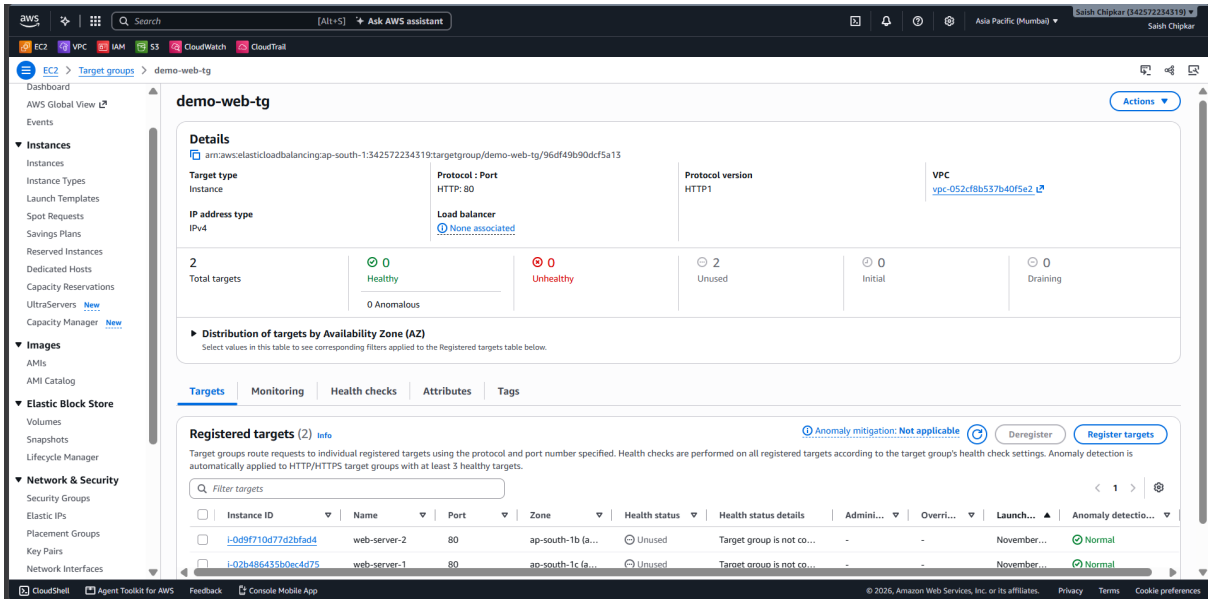


Figure 17: Opening the Load Balancers section and starting load balancer creation.

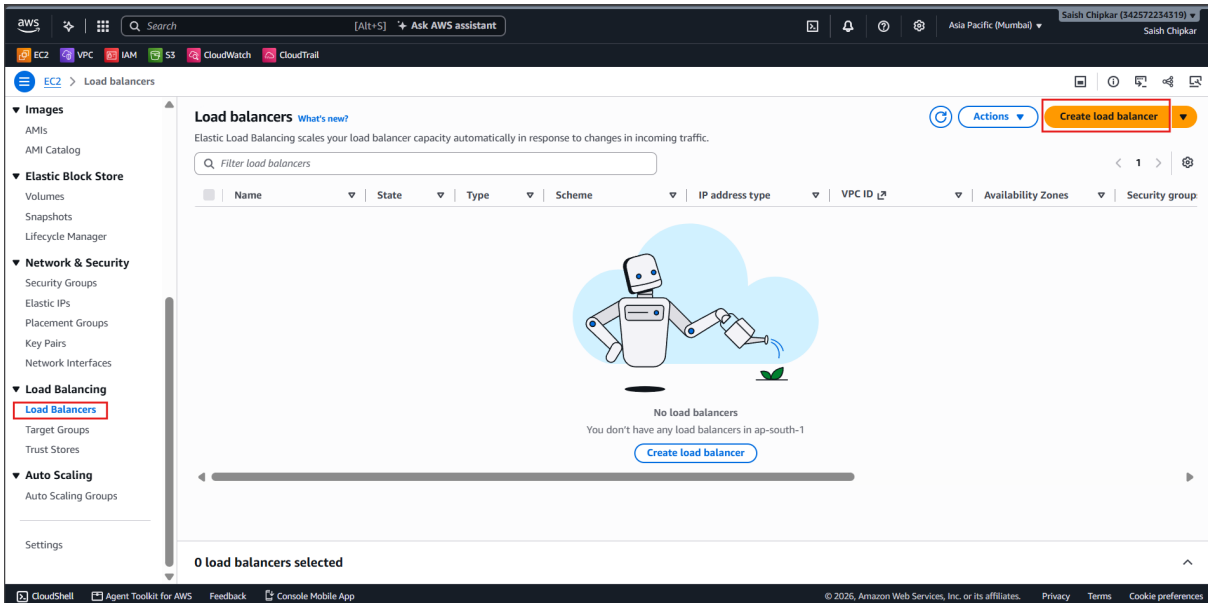


Figure 18: Selecting Application Load Balancer from the available load balancer types.

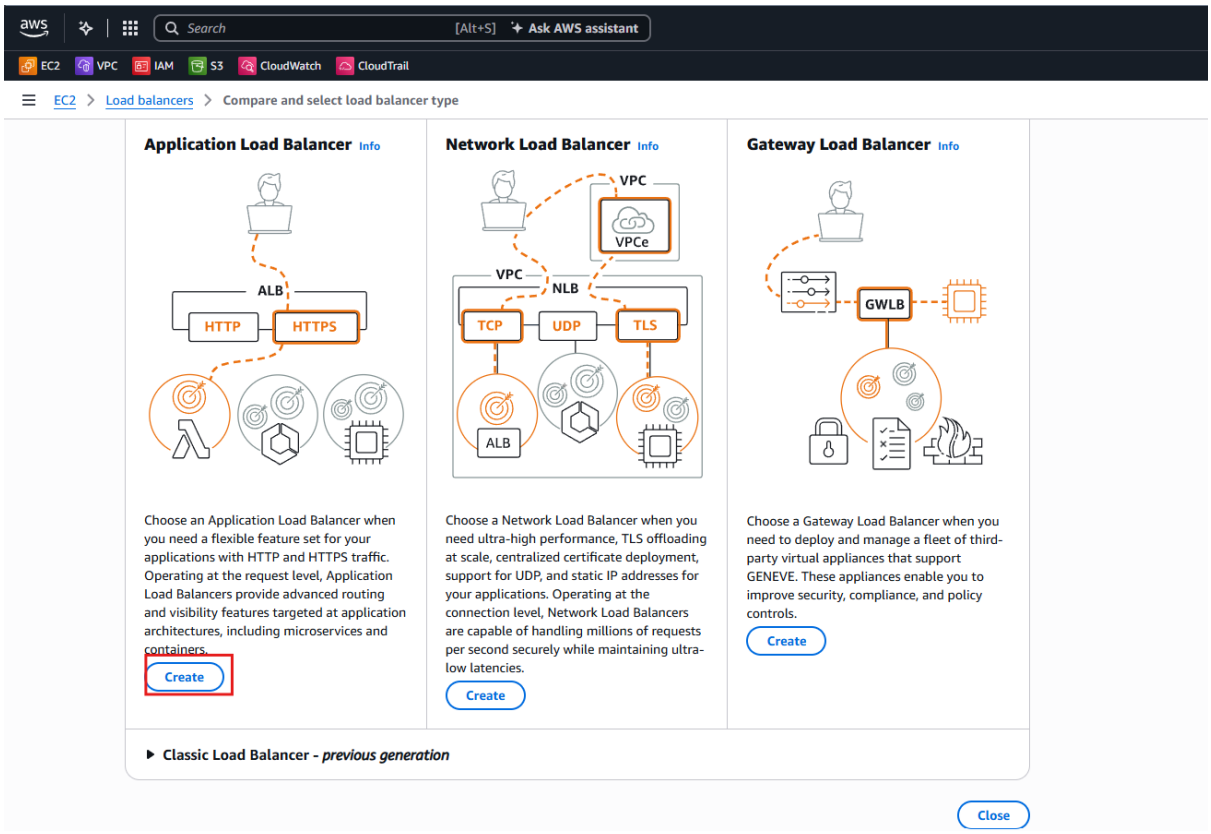


Figure 19: AWS comparison of load balancer types; Application Load Balancer is selected for HTTP application traffic.

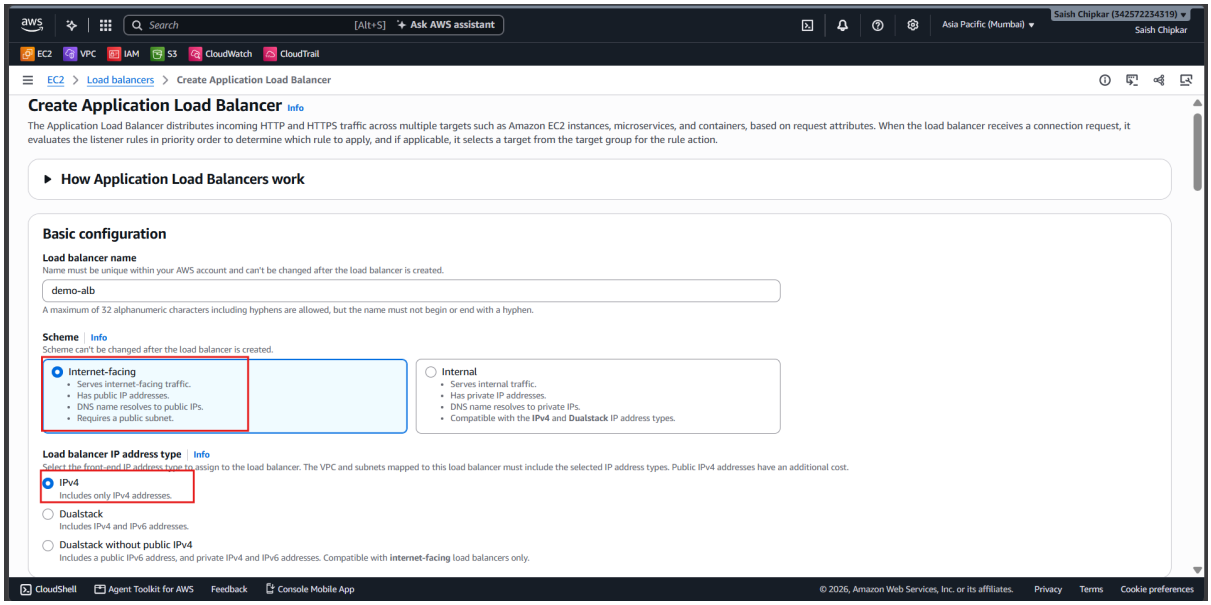


Figure 20: Configuring the ALB name, internet-facing scheme, and IPv4 address type.

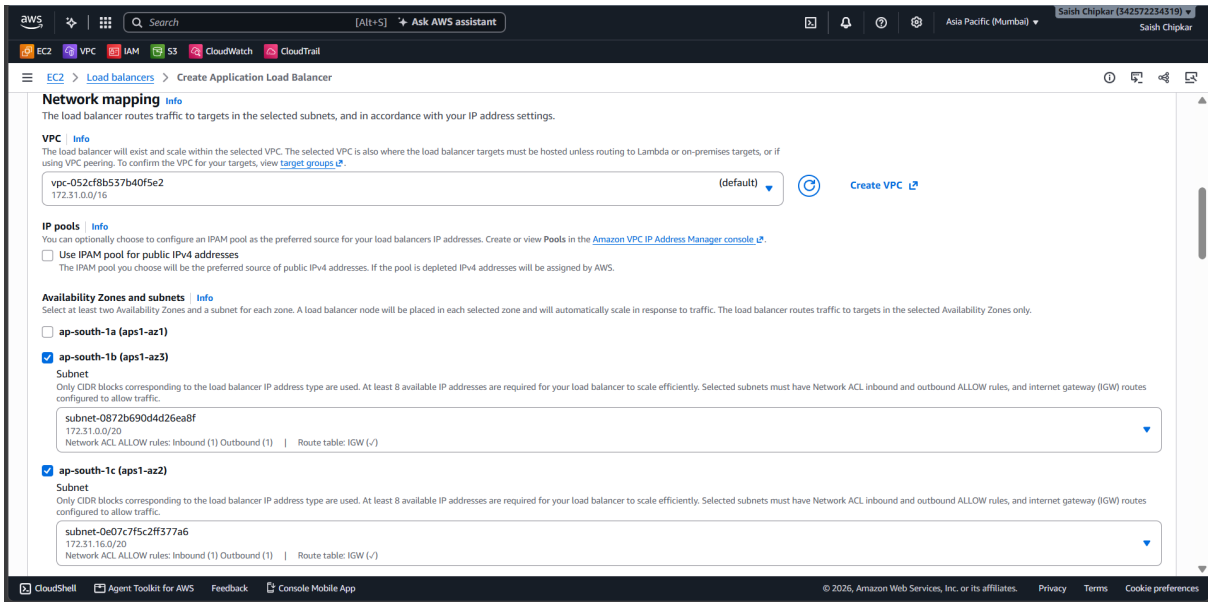


Figure 21: Selecting the VPC and subnets in two Availability Zones.

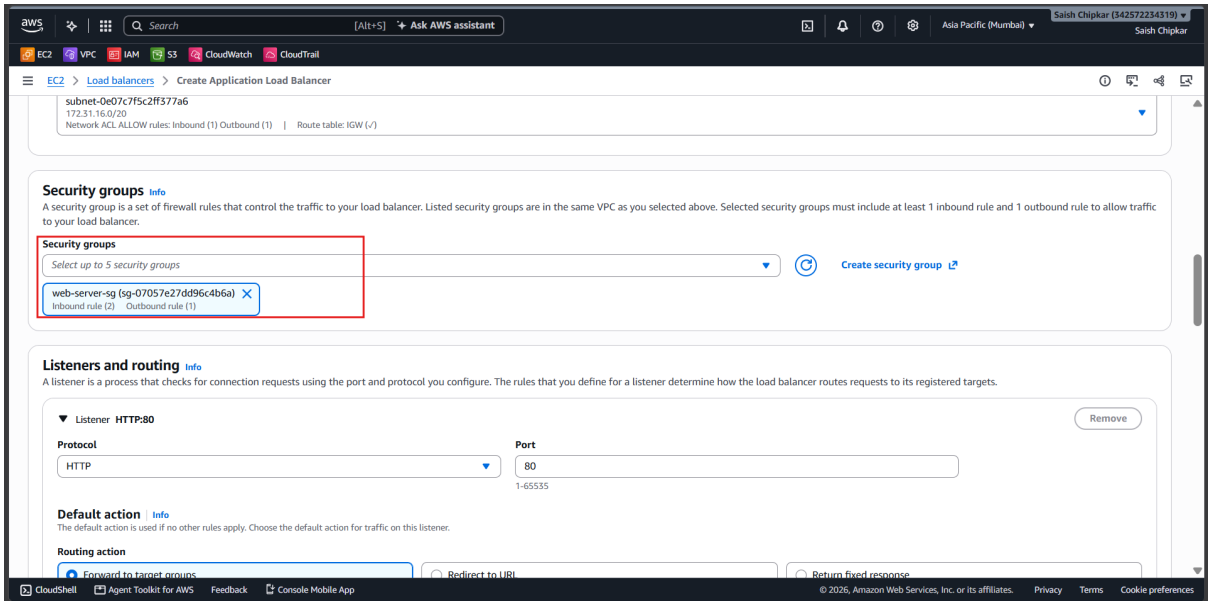


Figure 22: Configuring the load balancer security group and HTTP listener.

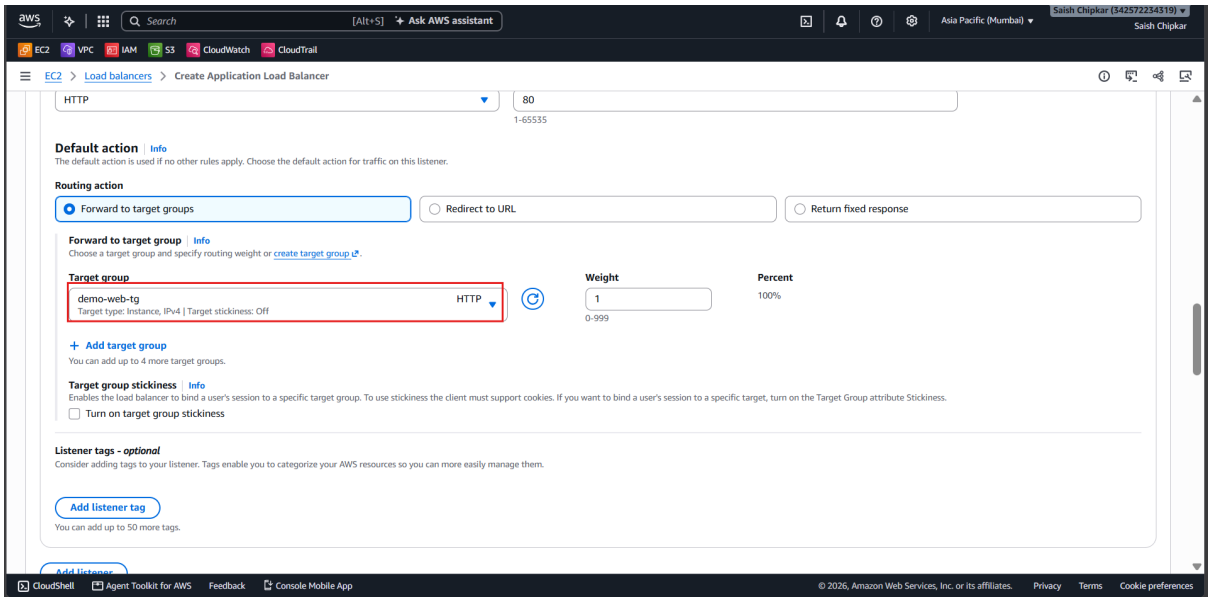


Figure 23: Setting the listener default action to forward requests to `demo-web-tg`.

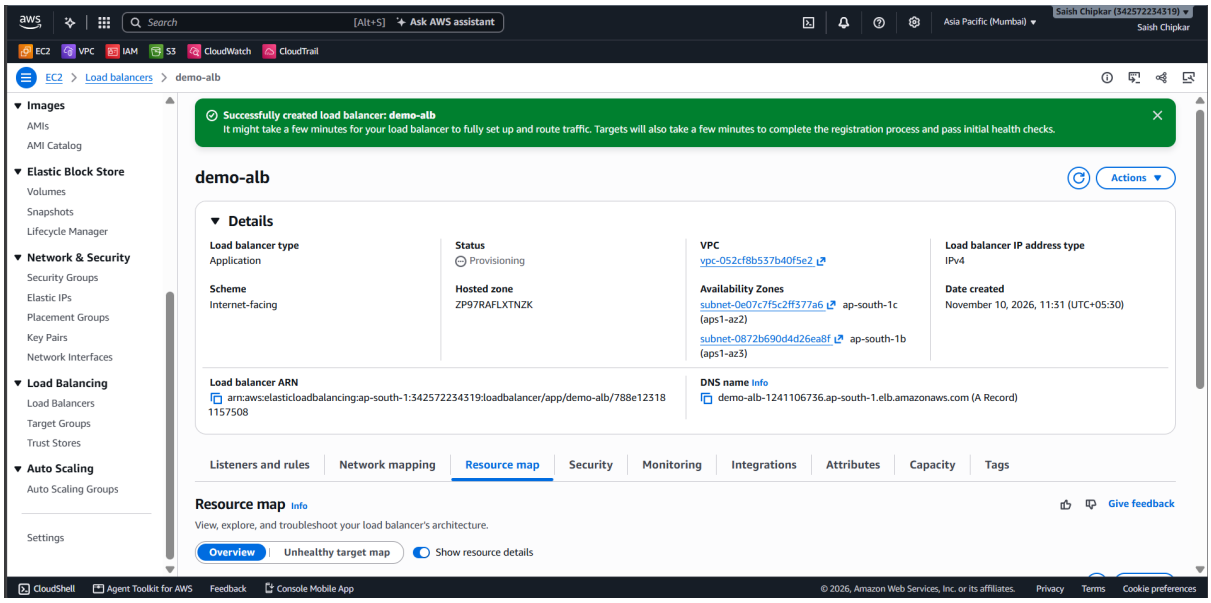


Figure 24: Successful creation of the `demo-alb` load balancer and display of its DNS name.

Security group verification: The ALB must be associated with `alb-sg`, while the EC2 instances must be associated with `web-server-sg`. The backend security group's HTTP inbound source must be the ALB security group. Verify these associations before testing.

4.7 Step 7: Wait for Target Health Checks

1. Open the `demo-web-tg` target group.

2. Check the **Targets** tab.
3. Wait until both registered EC2 instances show the status **Healthy**.
4. If a target remains unhealthy, verify that Nginx is running, the health check path is `/`, the instance security group allows HTTP from `alb-sg`, and the target group uses port 80.

4.8 Step 8: Test the Load Balancer

1. Copy the DNS name from the `demo-alb` details page.
2. Open the DNS name in a browser using HTTP, or run repeated requests using `curl`.
3. Each response should identify one of the backend servers.

For Windows Command Prompt, replace the placeholder with the actual ALB DNS name:

Listing 3: Sending ten HTTP requests from Windows Command Prompt

```
for /l %i in (1,1,10) do @curl -s http://<YOUR-ALB-DNS-NAME> && echo.
```

For Linux, macOS, or Git Bash:

Listing 4: Sending ten HTTP requests from a Unix-like terminal

```
for i in {1..10}; do
    curl -s http://<YOUR-ALB-DNS-NAME>
    echo
done
```

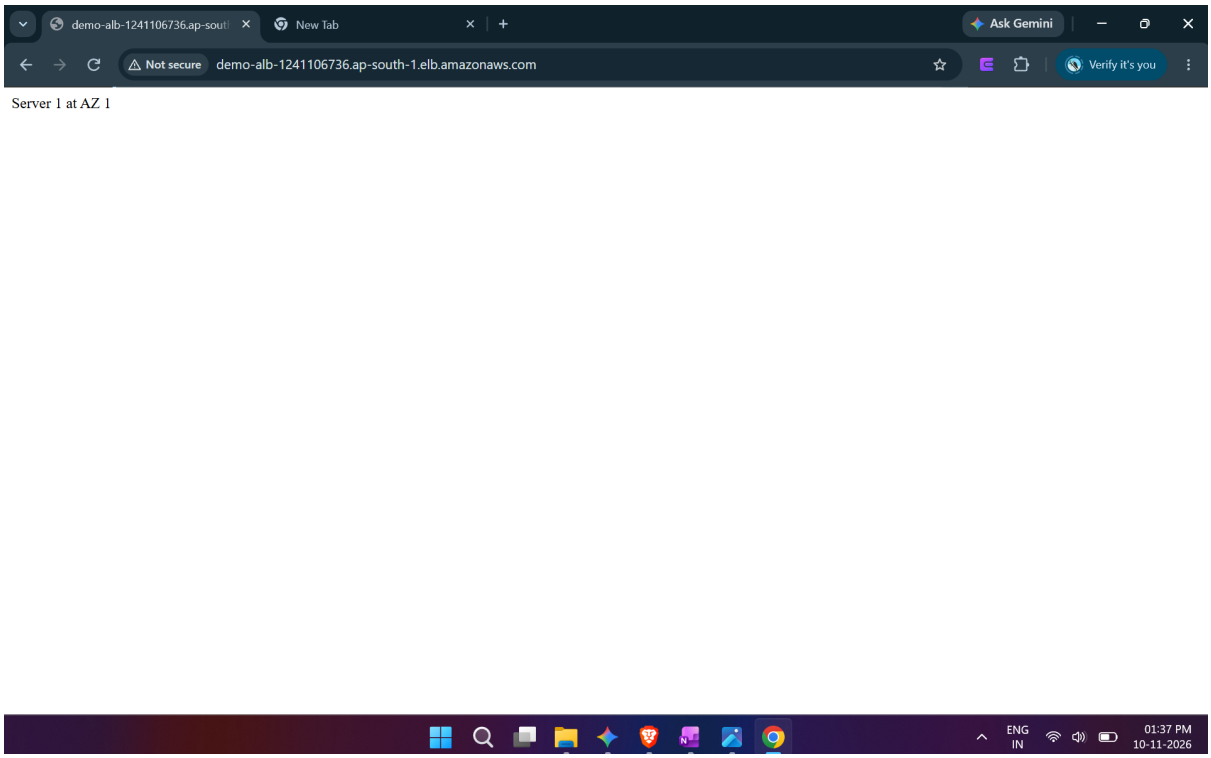


Figure 25: Browser response served by the first backend web server through the ALB DNS name.



Figure 26: Browser response served by the second backend web server through the same ALB DNS name.

5. Result

Two Amazon Linux 2023 EC2 web servers were deployed in separate Availability Zones and registered in the target group `demo-web-tg`. An internet-facing Application Load Balancer named `demo-alb` was created with an HTTP listener on port 80 and configured to forward requests to the target group. Accessing the ALB DNS name returned distinct responses from the two backend web servers, demonstrating traffic distribution through the load balancer.

6. Precautions

1. Deploy the backend instances in different Availability Zones for improved availability.
2. Use separate security groups for the ALB and backend instances.
3. Allow inbound HTTP traffic to the EC2 instances only from the ALB security group.
4. Restrict SSH access to a trusted administrator IP address.
5. Ensure that the ALB subnets have the required Internet Gateway route for an internet-facing load balancer.
6. Wait for both targets to become healthy before performing the request-distribution test.
7. Do not assume that every short test will show a perfectly alternating sequence; confirm that responses from both targets are received.
8. Delete the ALB, target group, and EC2 instances after the practical if they are no longer required, to avoid ongoing AWS charges.

7. Conclusion

The practical successfully demonstrated how an AWS Application Load Balancer distributes HTTP requests across multiple EC2 web servers. The target group provided backend registration and health monitoring, while the ALB listener accepted client requests and forwarded them to healthy targets. Using instances in separate Availability Zones improves the resilience of the application compared with relying on a single web server.